If you don't want to define a separate route for every single post, you can use a `[slug]` segment within the route path (e.g., `app/blog/[post]/page.js`). This `[slug]` acts as a placeholder for the dynamic part of the URL, which could be anything like "my-first-post" or "welcome-message."

When a user visits a URL with a dynamic segment, Next.js captures that segment value and makes it available to their React component through a `params` prop. This `params` prop will be an object containing the extracted data from the dynamic segment (e.g., `{slug: "my-first-post"}`).

With this in mind, let's review some key directories and files within our Next.js project:

- public—This directory holds static assets like images, favicons, and any files directly accessible by the browser.
- src—This is the heart of our application, containing all the source code for components, pages, layouts, and functionalities:
  - app—This is a key folder that hosts the pages of the application.
  - components—This directory houses reusable UI components like buttons, cards, chat elements, and navigation bars.
  - hooks—This directory stores custom React hooks that encapsulate reusable logic and state management for components.
  - lib—Utility functions and helper code reside in this directory, promoting code organization and reuse.
  - app/(chat)—This looks like a new type of folder wrapped with parentheses. This is called a route group, and I explain how it works and why we use this in section 2.8.6.
  - layout.js—A layout.js file within a folder in the app directory serves as a layout component for that specific route segment. I explain how layout components work and why we use them in section 2.8.4.

NOTE While the projects use the src directory to house the application's source code, it's important to note that Next.js offers flexibility in this regard. Next.js also looks for the app folder at the top level of your project directory if there's no src directory. However, there's one key requirement: the folder must be named app. This flexibility allows developers to customize the project structure to their preferences and existing conventions.

2.8.2 Configuration

As you explore the inner workings of the Next.js project, you also encounter a new file: next.config.mjs. This file acts as the central configuration hub for your Next.js application, allowing you to customize various aspects of its behavior.

You might notice that our next.config.mjs file is currently empty. This doesn't mean it's not important—it simply means that for our current implementation, we don't require any special configuration options offered by Next.js.